

code, circuits, & construction

code and fabrication resources for physical computing and networking

tigoe.net | [pcomp home](#) | [blog](#) | [code, circuits, & construction](#) | [my del.icio.us links](#)

HOME

PAGES

About this site
All About
Microcontrollers
Understanding
Electricity
 Breadboards
Input & Output
 Digital Input & Output
 Analog Input
 Analog Output
 Controlling high-
 current circuits
Serial Communication
 Interpreting Serial
 Data
 MIDI
Motors
 Controlling DC Motors

CATEGORIES

arduino/wiring
AVR
BX-24
circuits
construction
java
Lantronix
Lingo
Max/MSP
OSX
pBasic (Basic stamp)
Perl
PHP
PIC
PicBasic Pro

Serial Communication

Serial Communication

In order to make two devices communicate, whether they are desktop computers, microcontrollers, or any other form of integrated circuit, we need a method of communication and an agreed-upon language. The most common form of communication between electronic devices is *serial communication*.

Communicating serially involves sending a series of digital pulses back and forth between devices at a mutually agreed-upon rate. The sender sends pulses representing the data to be sent at the agreed-upon *data rate*, and the receiver listens for pulses at that same rate. This is what's known as *asynchronous serial communication*. There isn't one common clock in asynchronous serial communication; instead, both devices have their own clock and agree on a rate to which to set their clocks.

For example, let's say two devices are to exchange data at a rate of 9600 bits per second. First, we would make three connections between the two devices:

- a common ground connection, so both devices have a common reference point to measure voltage by;
- one wire for the sender to send data to the receiver on (transmit line for the sender);
- one wire for the receiver to send data to the sender on (receive line for the sender).

Now, since the data rate is 9600 bits per second (sometimes called 9600 *baud*), the receiver will continually read the voltage that the sender is putting out, and every 1/9600th of a second, it will interpret that voltage as a new bit of data. If the voltage is high (+5V in the case of Wiring/Arduino, the PIC, and BX-

Processing
XBee

RSS FEEDS

Posts
Comments

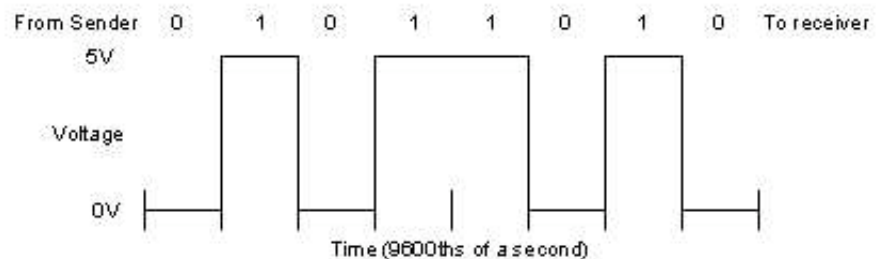
META

Log in

SEARCH

24), it will interpret that bit of data as a 1. If it is low (0V in the case of Wiring/Arduino, the PIC, and BX-24), it will interpret that bit of data as a 0. By interpreting several bits of data over time, the receiver can get a detailed message from the sender. at 9600 baud, for example, 1200 bytes of data can be exchanged in one second. If you have a home computer and a modem, you've seen serial communication in action. Your computer's modem exchanges information with your service provider's modem serially.

Let's look at a byte of data being exchanged. Imagine I want to send the number 90 from one device to another. First, I have to convert the number from the decimal representation 90 to a binary representation. in binary, 90 is 01011010. So my sending device will pulse its transmit line as follows:



As you can tell from this diagram, both devices also have to agree on the order of the bits. Usually the sender sends the highest bit (or *most significant bit*) first in time, and the lowest (or *least significant bit*) last in time. As long as we have an agreed upon voltage, data rate, and order of interpretation of bits, we can exchange any data we want serially.

For the data transmission above, a high voltage indicates a bit value of 1, and a low voltage indicates a voltage of 0. This is known as *true* logic. Many serial protocols use *inverted* logic, meaning that a high voltage indicates a logic 0, and a low voltage indicates a logic 1. It's important to know whether your protocol is true or inverted. For example, RS-232, described below, uses inverted logic.

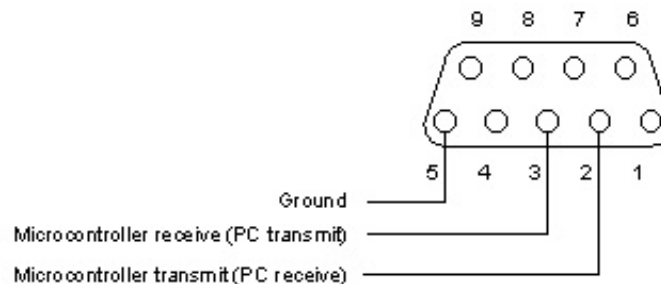
For most of our work, we'll be using a particular serial protocol called RS-232. The RS-232 standard defines voltages and general baud rate ranges for serial communications between devices using it. We won't be getting the voltages exactly right, but for most

applications, we'll be close enough. Until recently, most desktop computers had an RS-232 or similar serial port. Now, many desktop computers are shifting to other forms of serial communication such as USB, or Universal Serial Bus, and Firewire, which allow for more flexible configurations and faster data rates. the RS-232 standard is still very common in other devices, though, as it is cheaper to use than USB, simpler to implement, consumes less power, and provides more than adequate speeds for exchanging control data (i.e. data that allows one device to control another).

Serial Communication to a PC or Mac

The RS-232 serial ports on Windows-based PC's looks like this:

PC serial cable (facing the soldering lugs of a female connector)

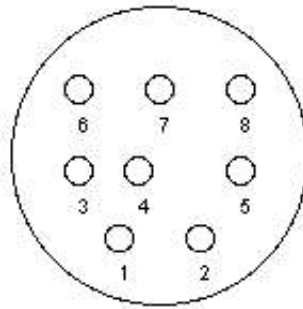


The connections for this are the same as the connector used to program the BX-24, and to debug from both the PIC and the BX-24, so you can use the same connector, or make a second one just like it. From there, you will need a 9-pin serial cable. Use any serial port you want, but be sure that all other programs that might be trying to use the serial port are turned off.

Wiring and Arduino boards have a built-in USB-to-serial converter, so they communicate serially using the USB port.

Macintoshes used to have two serial ports, one for the printer, one for the modem. The current Mac models (and increasingly, many PCs) do not come with a built-in serial port, and one must be installed. An RS-232 to USB serial port adaptor such as the [Keyspan](#) USB USA19HS Adaptor will do fine.

Classic Mac serial cable (Facing the male pins of a cable)



Mac 9-pin serial cable
Facing a male connector

Pin 3: Mac Transmit to Microcontroller receive
Pin 4: Ground to Microcontroller Ground
Pin 5: Mac Receive to Microcontroller transmit

Serial Output from the Microcontroller

Once you've got the computer and the microcontroller connected, you'll need to write a program to address the serial ports. The process is slightly different on the different microcontrollers, but there are some elements common to all of them.

Desktop computers have a place they store incoming data from the serial port, called a *serial buffer*. It's a little area of memory to store whatever comes in the serial port. Because of this, they can do other tasks while waiting for data to come in, and act on the data from the buffer. Microcontrollers usually don't have a serial buffer, but the BX-24 can have a small one, as do Wiring and Arduino. Details on how to set it up on the BX-24 follow. It's automatically set up for you on a Wiring or Arduino module. On the PIC, you don't have a serial buffer, so you don't have to set it up.

Multimedia computers have one or more serial ports, and each port can be controlled by only one program at a time. To use a port, you have to open it, set its parameters, and then look for data. On the PIC, there are no serial ports. Each serial command defines the parameters of communication within the command. On the BX-24, you have to define the port, open it, then look for data, as on the PC.

On Wiring or Arduino boards, the serial pins are fixed, and you can't change them. You can, however, use the [software serial library for Arduino](#) if you need to use different pins, or if you need more than one serial port.

See the [PIC serial notes](#) for the software details of serial output using PicBasic Pro. See the [BX-24 serial notes](#) for the software details using a BX-24. See the [Arduino serial lab](#) for more on serial on Arduino.

If you're sending a single byte out serially, it's easy for the receiving device to know what's coming. It will read a byte and know the value. Sometimes, however, you have to send multiple bytes to get a message across. Those bytes have to be reassembled on the receiving side. More on that in the [serial data interpretation notes](#).



(cc) tigoe | Thanks, [WordPress](#) | [Barthelme](#) theme by [Scott](#) | Valid [XHTML](#) & [CSS](#) | [RSS](#):
[Posts](#) & [Comments](#)